

Funciones

Programación I

UNRN

Universidad Nacional
de Río Negro



Programación I

Clase 04: Funciones

Universidad Nacional de Río Negro | Ingeniería en Sistemas | Ciclo 2026

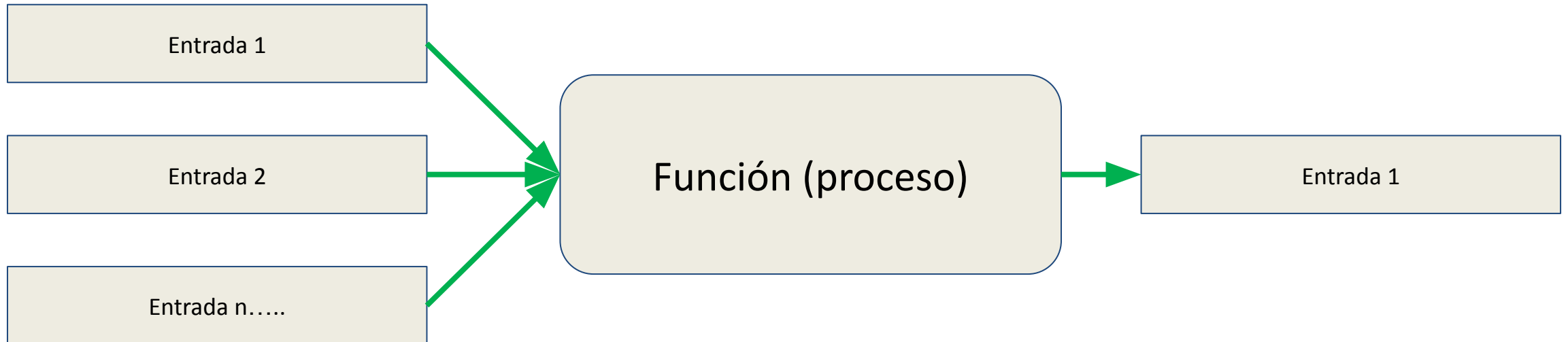
Prof. Daniel Teira/Diego Diaz

Sede Bariloche

1. Definición de función

“Una función es un bloque de código independiente y reutilizable diseñado para realizar una tarea específica dentro de un programa.”

En lugar de escribir las mismas instrucciones muchas veces, el código se empaqueta bajo un nombre y se puede "llamar" o ejecutar cada vez que se necesite.



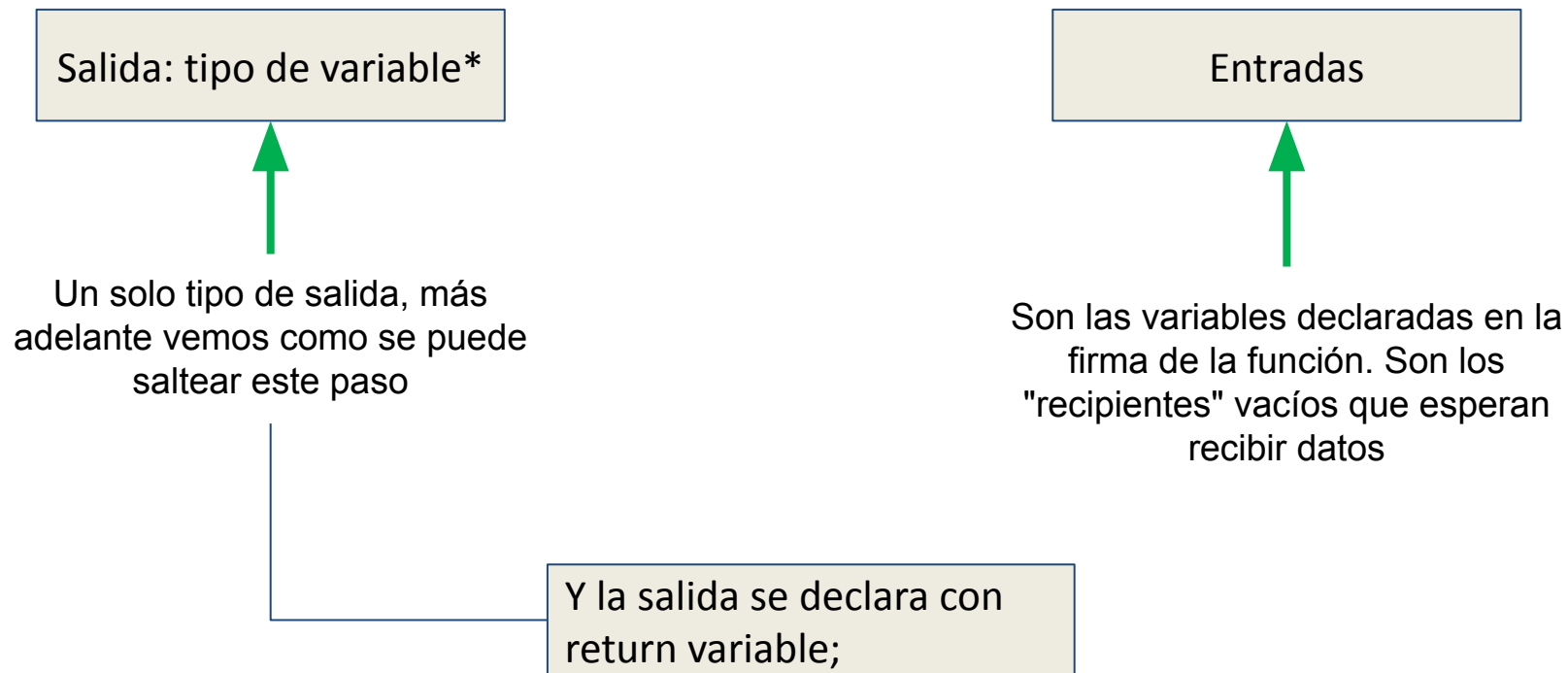
¿Qué recibe? (Entrada): Los argumentos o parámetros, que son los datos que la función necesita para operar. (Dominio)

¿Qué hace? (Proceso): El cuerpo de la función, delimitado entre llaves {}, donde se ejecutan las instrucciones.

¿Qué devuelve? (Salida): El valor de retorno, que es el resultado final que la función envía de vuelta mediante la palabra clave return. (imagen)

1. Definición de función:

tipo Nombre de la función(tipo nombre, ...)



2. Ejemplo de función: calculadora de área

```
#include <stdio.h>
```

```
float calcularArea(float base, float altura)
{
    float area = base * altura;
    return area;
}
```

Definición de la función

Las variables “*base*” y “*altura*” solo viven dentro de la función, una vez que es calculada “*área*”, estas son destruidas.

```
int main() {
    float baseRectangulo = 5.5;
    float alturaRectangulo = 3.0;
    float resultado;

    printf("--- Cálculo de Área de un Rectángulo ---\n");

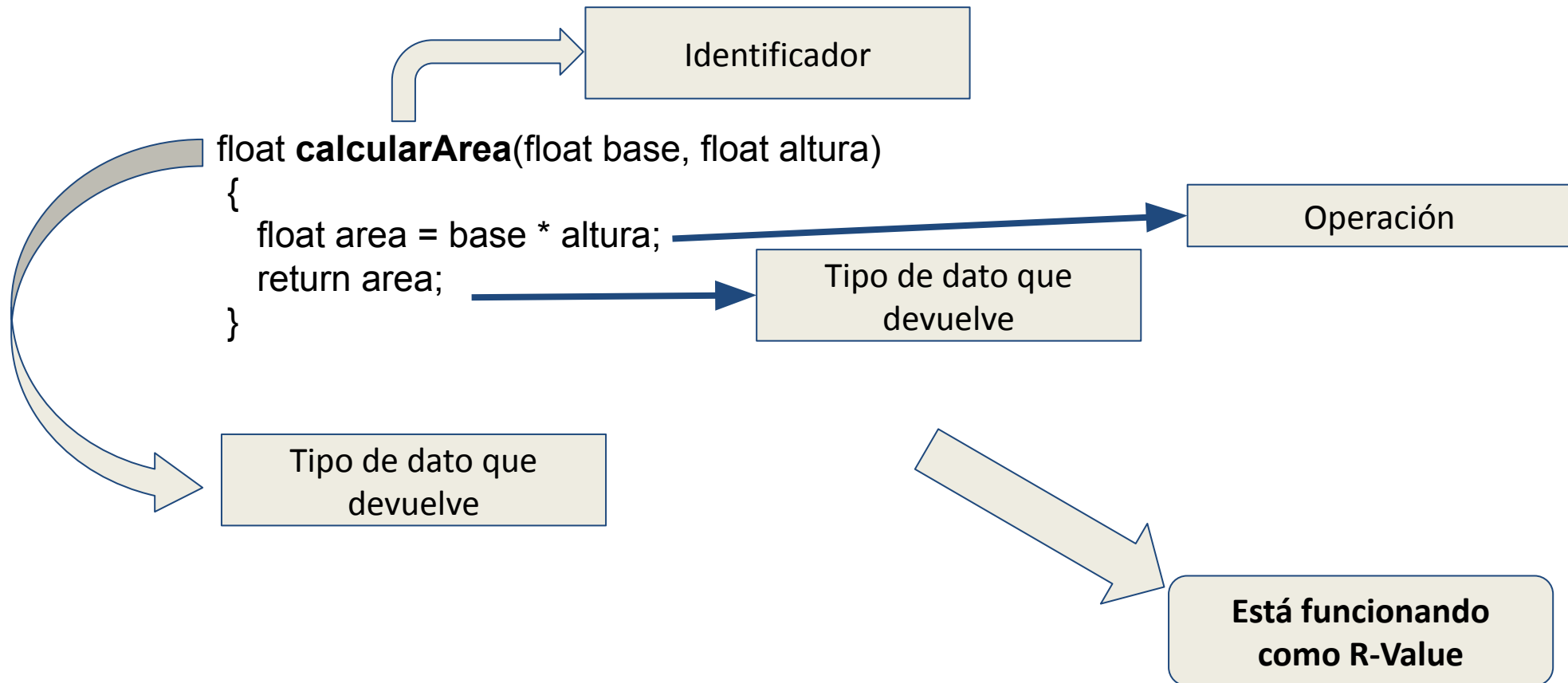
    resultado = calcularArea(baseRectangulo, alturaRectangulo);

    printf("Base: %.2f\n", baseRectangulo);
    printf("Altura: %.2f\n", alturaRectangulo);
    printf("El área calculada es: %.2f\n", resultado);

    return 0;
}
```

Definición de la función

2. Ejemplo de función: calculadora de área



3. Formas básicas para llamar a una función:

```
int resultado = suma(10, 45);  
int plus = suma(suma(10,20), 10);  
int otro = suma(resultado, 10);
```



Como puedo anidar funciones y puedo pasar como argumento una función, siempre y cuando defina bien las salidas y entradas

```
void mi_funcion()  
{  
    bloque;  
}
```



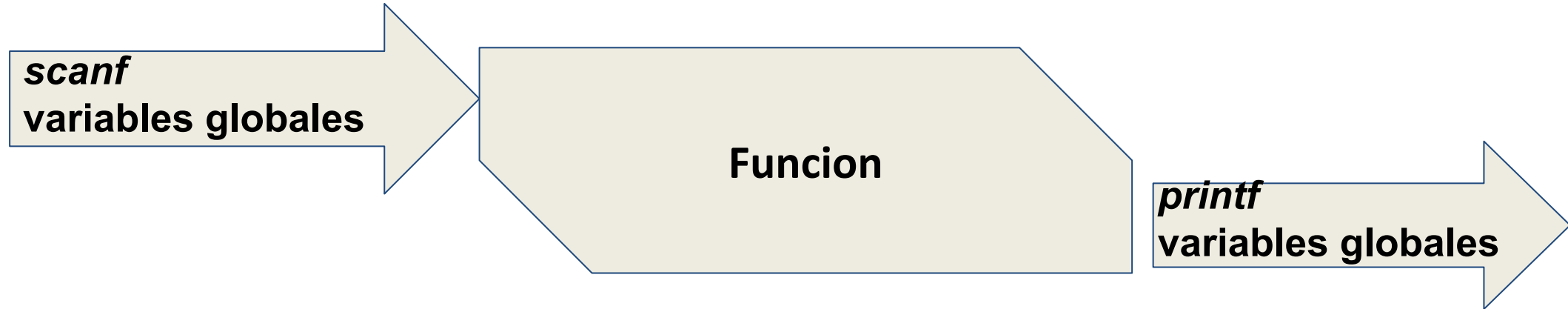
Como puedo anidar funciones y puedo pasar como argumento una función, siempre y cuando defina bien las salidas y entradas

```
void mi_funcion()  
{  
    return;  
}
```

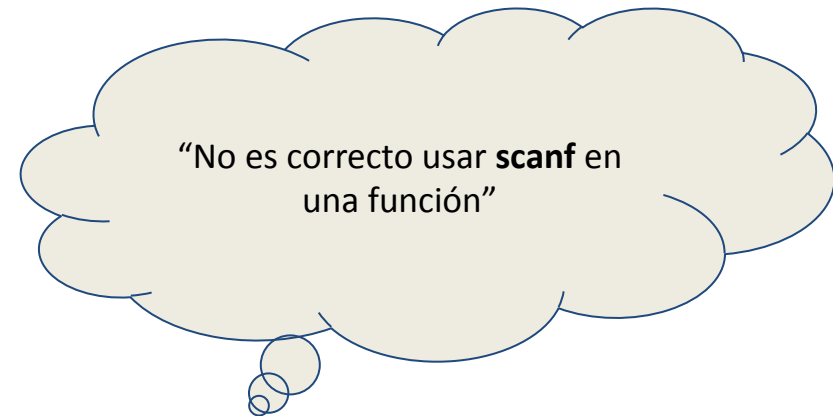


O por ejemplo una función que no tiene argumentos, pero sí return.

3. Cual es la aplicación de una función que no tiene argumentos y no devuelve nada?



```
int leer_entero()
{
    int valor = 0;
    scanf("pasame un valor entero %d", &valor);
    return valor;
}
void mostrar_temperatura(double celcius)...
```



4. Variables globales vs variable locales, como se entiende en el contexto de funciones:

Variable Local *(El secreto de la función)*

- **Dónde vive (Alcance):** Solo existe y es conocida dentro de las llaves `{ }` donde fue declarada. Las variables creadas dentro de una función no existen fuera de ella (en el main u otras funciones).
- **Cuándo muere (Tiempo de vida):** Nace cuando la función empieza a ejecutarse y se destruye automáticamente en cuanto la función termina y devuelve el control.
- **Ventaja:** Son seguras. Nadie desde afuera puede modificarlas por accidente, lo que hace que tu código sea predecible.

```
#include <stdio.h>
```

```
void calcular()
{
    int resultado = 100; // Variable LOCAL a la función calcular()
    printf("El resultado es: %d\n", resultado); // Funciona perfecto aquí adentro
}
```

```
int main()
{
    calcular();
    printf("%d", resultado); //: El main no tiene idea de que es "resultado"
    return 0;
}
```

4. Variables globales vs variable locales, como se entiende en el contexto de funciones:

Variable Global (El megáfono del programa)

- **Dónde vive (Alcance):** Es completamente pública. Puede ser vista, leída y modificada por cualquier función del archivo.
- **Cuándo muere (Tiempo de vida):** Vive durante todo el tiempo que el programa esté en ejecución.
- **El peligro:** Como cualquiera puede cambiarla, si tu programa falla porque el valor es incorrecto, es muy difícil rastrear qué función exacta modificó la variable y causó el error.

```
#include <stdio.h>
```

```
int puntaje = 0; // Variable GLOBAL
```

```
void ganarPuntos() {  
    puntaje = puntaje + 10; // Modifica la variable global directamente  
}
```

```
int main() {  
    ganarPuntos();  
    printf("Puntaje total: %d\n", puntaje); // El main también la lee y muestra 10  
    return 0;  
}
```

4. Ilustración de variable global vs local:

- **Variable Global (La pizarron del aula):** Está al frente de la clase. Cualquier persona en el salón puede ver lo que está escrito, y cualquiera puede pasar al frente con un borrador y cambiar la información para todos.
- **Variable Local (Tu cuaderno personal):** Está en tu escritorio. Solo vos puedes escribir, leer y usar su información. Si el alumno de al lado necesita un dato de tu libreta, no puede agarrarla; se lo tienes que pasar explícitamente vos a él (esto equivale a pasar un parámetro a otra función).

4. variable tipo Static (uso en funciones):

Conservan su valor entre distintas llamadas a la función. Se inicializan solo una vez y su valor no se destruye al salir de la función.

Uso típico: Contadores, acumuladores o almacenamiento en caché.

Si declaro una variable global como *static* sólo puede ser modificada en el contexto de main.c

```
#include <stdio.h>
```

```
void incrementar()  
{  
    static int contador = 0; // Se inicializa sólo la primera vez  
    contador++;  
    printf("Contador: %d\n", contador);  
}
```

```
int main()  
{  
    incrementar(); // Imprime: Contador: 1  
    incrementar(); // Imprime: Contador: 2  
    return 0;  
}
```

Tiene ciertas limitaciones, tales com:

- **Pérdida de historial:** No puedes mantener un rastro independiente para cada llamada recursiva porque el espacio de memoria está unificado.
- Si una función que usa una variable `static` se llama a sí misma de forma recursiva, cada instancia modificará el mismo valor.
- No puedo hacer esto: `static int mi_var = calcular_valor();`

4. Pasaje de argumentos:

```
#include <stdio.h>
```

```
int incremento(int variable)
{
    variable = variable+ 1;
    return variable;
}
```

```
int main()
{
    int variable = 10;
    incremento(variable);
    printf("variable: %d\n", variable);
    return 0;
}
```

Que imprimen ambos ejemplos?

Lo que pasa en la función, queda en la función!

```
#include <stdio.h>
```

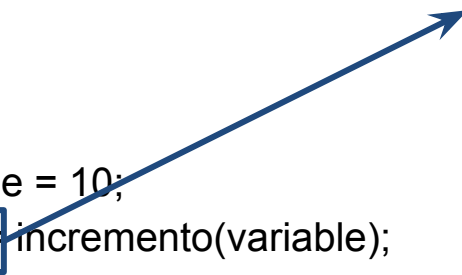
```
int incremento(int variable)
{
    variable = variable+ 1;
}
```

```
int main()
{
    int variable = 10;
    variable = incremento(variable);
    printf("variable: %d\n", variable);
    return 0;
}
```

4. Pasaje de argumentos:

```
#include <stdio.h>
int incremento(int valor)
{
    valor = valor + 1;
    return valor;
}
int main()
{
    int variable = 10;
    variable = incremento(variable);
    printf("variable: %d\n", variable);
    return 0;
}
```

La unica manera es
guardar el valor de salida



5. Documentación: TIPS!

1. Escriban la documentación antes de programar.
2. Mantener la prosa incluyendo el identificador del argumento y de la función.
3. No repetir lo que está en el código (el tipo, el identificador, por ejemplo).
4. Cambien los identificadores si quedan en una mejor prosa al documentar
5. Mantener la documentación actualizada
6. Indiquen qué valores son (y no) válidos: Indiquen explícitamente todos los retornos 'especiales': -1 cuando...0 cuando esta todo bien (Divisor en 0 en división)
7. Describir la salida

5. Documentación: Descripción de la función

```
/*  
* Toda función debe tener un comentario que  
* describa que es lo que hace.  
* Indicando que rol cumple y hace cada argumento.  
* Esta función se encarga de sumar dos numeros enteros  
* @param termino1 es el primer termino de la suma  
* @param termino2 es el segundo termino a sumar  
* @returns la suma de ambos términos  
* PRE-CONDICION: numero enteros en el rango de +/-2^8.  
* POST-CONDICION: numero entero que es suma de los  
* términos de entrada. La salida tendrá  
* un rango entre +/- 2^9  
*/  
int suma(int termino1, int termino2);
```


6. Prototipo de una función

Un prototipo de función es la declaración formal de una función en C antes de que sea implementada o utilizada. Funciona como una plantilla que le avisa al compilador la existencia, el nombre y la estructura de la función:

- El prototipo especifica el tipo de retorno, el nombre de la función y los parámetros, terminando siempre con un punto y coma ;

```
tipo_de_retorno nombre_funcion(tipo_parametro1, tipo_parametro2);
```

```
#include <stdio.h>
```

```
// 1. PROTOTIPO (Declaración al inicio)
```

```
int sumar(int a, int b);
```

```
int main() {
```

```
    // 2. USO (El compilador ya sabe que 'sumar' existe)
```

```
    int resultado = sumar(5, 3);
```

```
    printf("Resultado: %d\n", resultado);
```

```
    return 0;
```

```
}
```

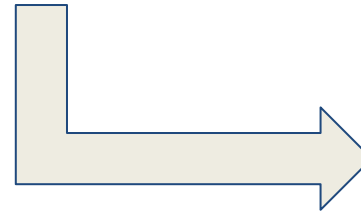
```
// 3. DEFINICIÓN (El código real de la función)
```

```
int sumar(int a, int b) {
```

```
    return a + b;
```

```
}
```

¿Por qué es necesario?



- **Evita errores de orden:** C lee el código de arriba hacia abajo. Si llamas a una función en la línea 20, pero su código real está escrito en la línea 50, el compilador arrojará un error. El prototipo en la línea 5 soluciona esto.
- **Verificación de tipos:** Permite al compilador revisar si estás pasando los argumentos correctos al llamar a la función. [\[1\]](#)

6. Prototipo de una función

```
#include <stdio.h>

// Aca defino la función completa:
void saludar() {
    printf("¡Hola desde la
función!\n");
}

int main() {
    saludar(); // El compilador ya la
conoce
    return 0;
}
```

```
#include <stdio.h>

void saludar(); // Prototipo(declaración)

int main() {
    saludar(); // El compilador sabe que existe y
qué esperar
    return 0;
}

// Definición completa de la función
void saludar() {
    printf("¡Hola desde abajo de main!\n");
}
```

7. Pre_condicion vs post_condicion:

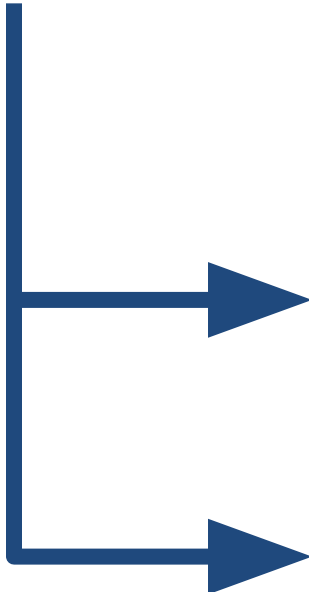
Una precondition es un requisito que debe ser verdadero antes de ejecutar una función, mientras que una postcondición es una garantía de lo que el código cumplirá al finalizar la ejecución

```
//funcion para dividir
int dividir(int dividendo, int divisor, int *resultado_salida)
{ // 1. PRECONDICIÓN: El divisor no puede ser cero.
  if (divisor == 0) {
    return ERROR_DIVISIÓN_POR_CERO; // Se aborta la operación de forma segura
  }

  // Operación principal
  int cociente = dividendo / divisor;
  int resto = dividendo % divisor;

  // 2. POSTCONDICIÓN: El resultado debe cumplir la propiedad matemática:
  // (cociente * divisor) + resto debe ser igual al dividendo original.
  if ((cociente * divisor) + resto != dividendo) {
    return ERROR_POSTCONDICION_FALLIDA; // El cálculo falló a nivel lógico o de hardware
  }

  // Si todo es correcto, guardamos el resultado en el puntero de salida
  *resultado_salida = cociente;
  return EXITO;
}
```



Precondición: Lo que se exige como entrada (por ejemplo, que un puntero no sea `NULL` o que un número sea positivo).

Postcondición: Lo que la función promete devolver o modificar al terminar de forma correcta (por ejemplo, un valor mayor a cero).

8. Resumen:

- Definición de función (ejemplos)
- Formas básicas para llamar a una función
- Variables globales vs variable locales
- variable tipo Static
- Pasaje de argumentos
- Documentación: TIPS
- Prototipo de una Función
- Pre_condicion vs post_condicion